

Custom Kernels for Clustering/Classification

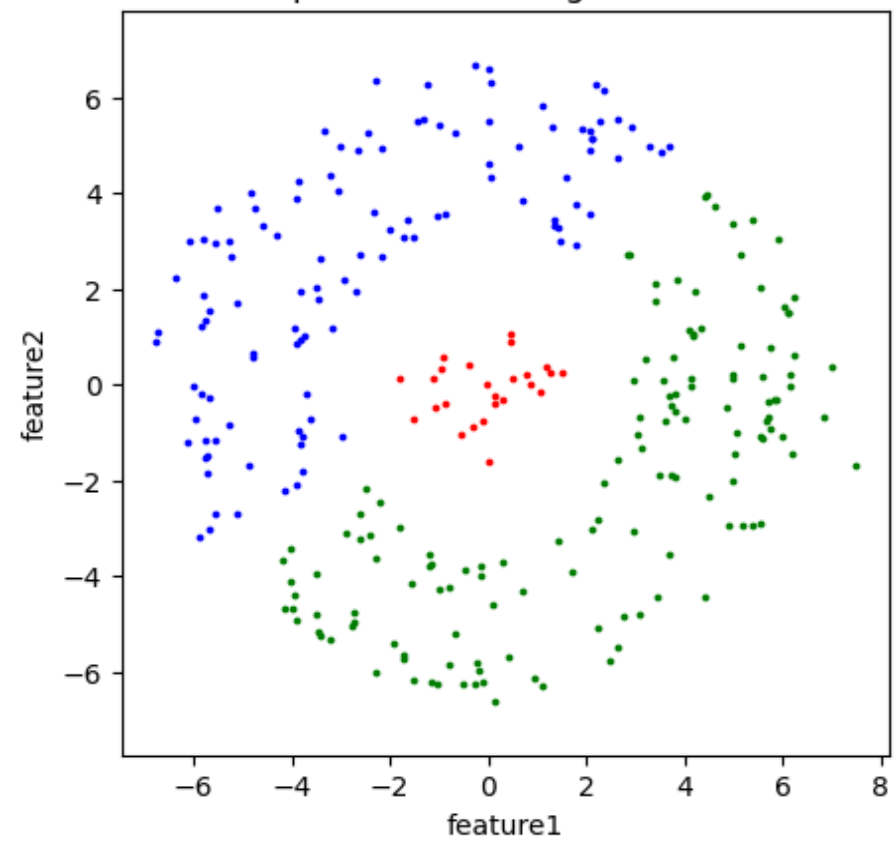
Adam Handwerger

Difficult for standard kernels when the number of clusters, $K \geq 3$

Example1: Clusters that form concentric hyperspheres. It would seem that the standard Gaussian Kernel should do the trick.

Unfortunately the cost function is actually minimized for a grouping like the following:

Kernel Spectral Clustering with RBF Kernel



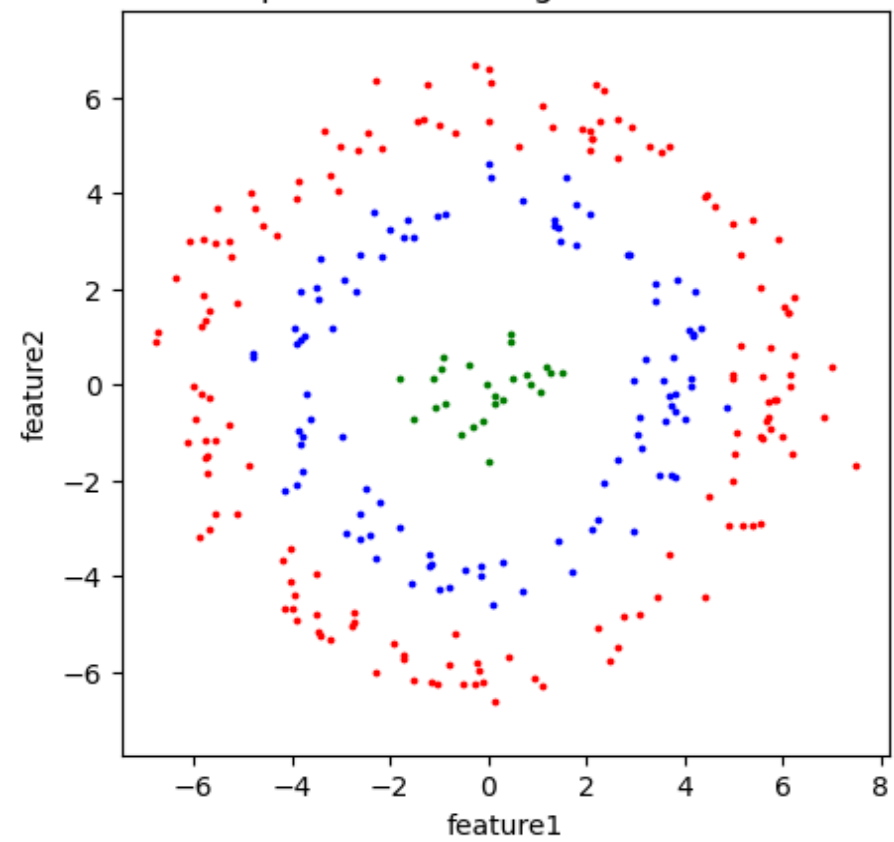
Based on the symmetry of the data set, let's develop a custom kernel to solve this problem.

The sklearn package has features that allows you use custom kernels in the spectral clustering algorithm. For this example let's use

$$k(x, y) = \exp(-\alpha|(\|x\| - \|y\|)|)$$

where α is an adjustable parameter. Choosing this kernel with $\alpha = 0.5$ gives a better grouping.

Kernel Spectral Clustering with Custom Kernel



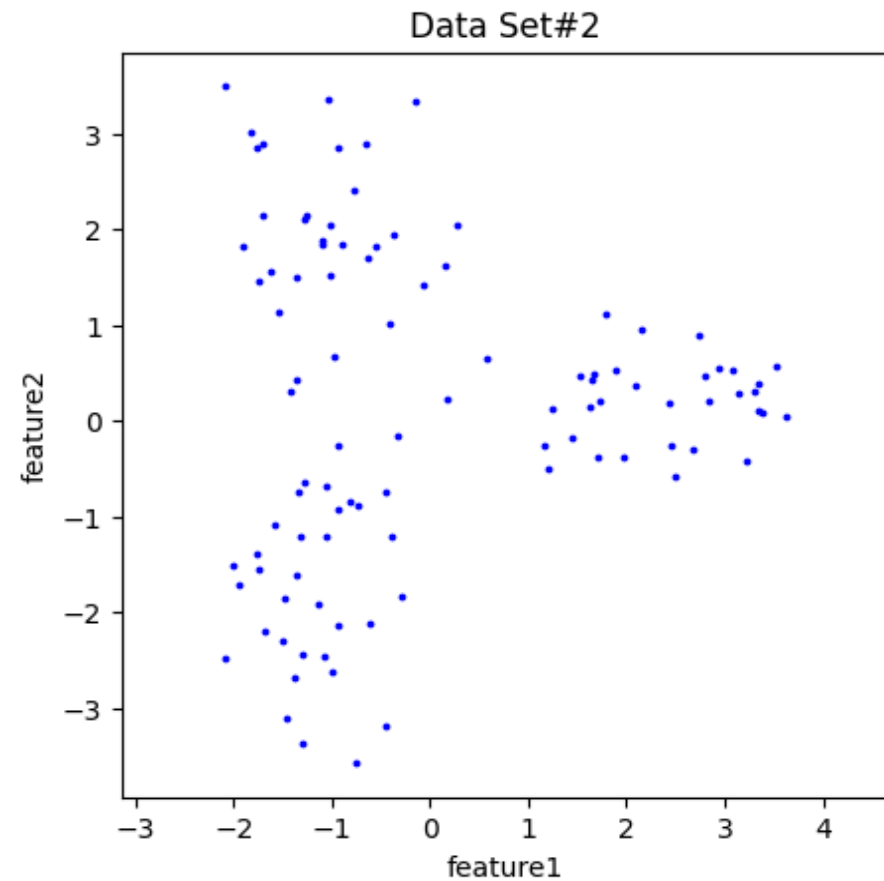
Finally check to see if k is p.d.

```
eigvalues,eigenvectors=np.linalg.eigh(affinity_matrix)
np.any(eigvalues<0)
```

Python

False

For the second example consider a data set with some angular symmetry.



Spectral Clustering gives a good grouping when the γ parameter is adjusted correctly using the 'rbf' affinity setting.

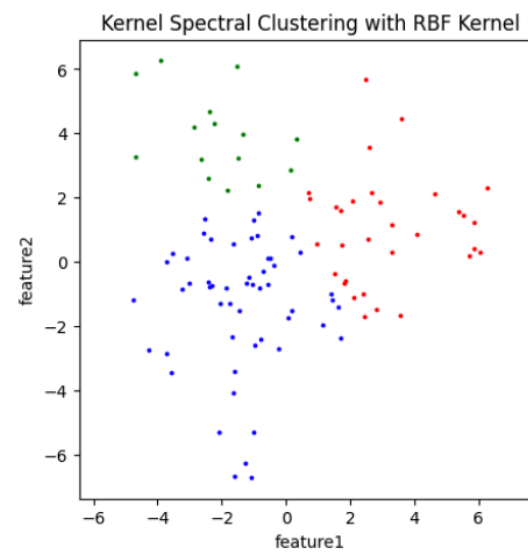
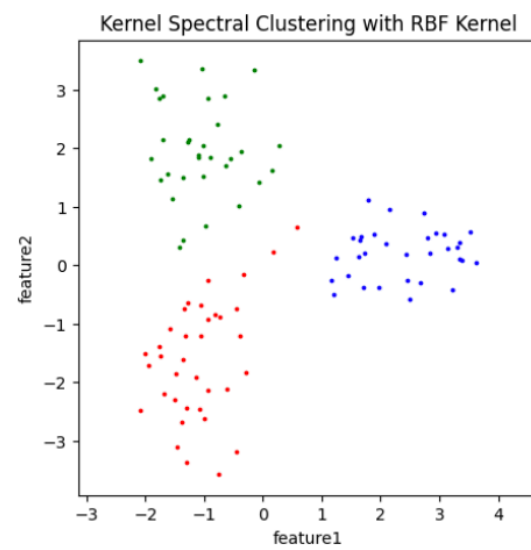
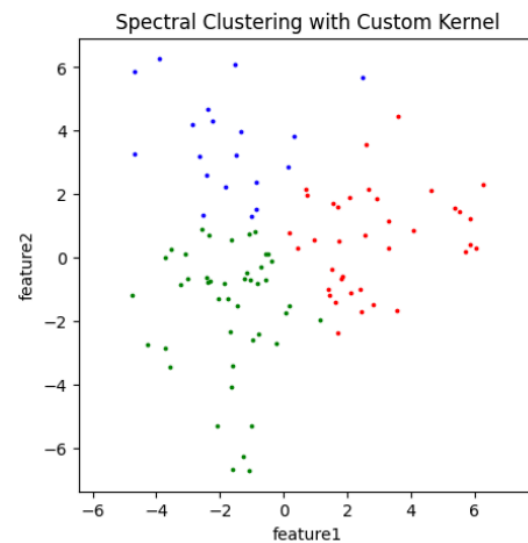
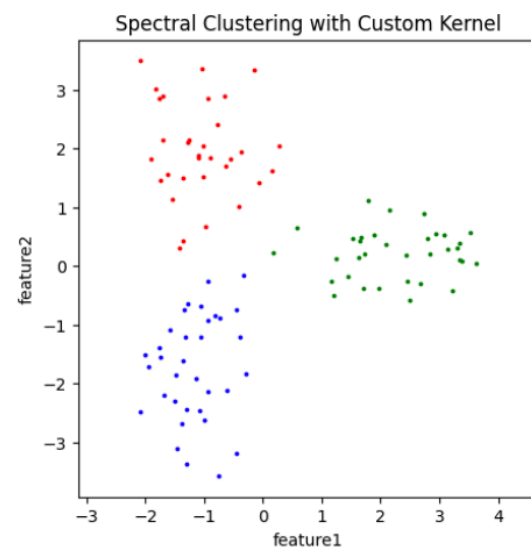
Unfortunately, γ is sensitive to noise.

When more noise was added to the data γ had to be adjusted from 10 to 0.1. See plots below.

Let's see if we can find a kernel that will give a good grouping that is not as sensitive to noise.

Consider the kernel,

$$k(x, y) = |\cos(\alpha ||x - y||)|$$



For the upper plots both have $\alpha = 0.1$ and for the lower plots $\gamma = 10$ and $\gamma = 0.1$ left to right.

Let's check if k is p.d.

```
eigvalues,eigenvector=np.linalg.eigh(affinity_matrix)
np.any(eigvalues<0)
```

Python

True

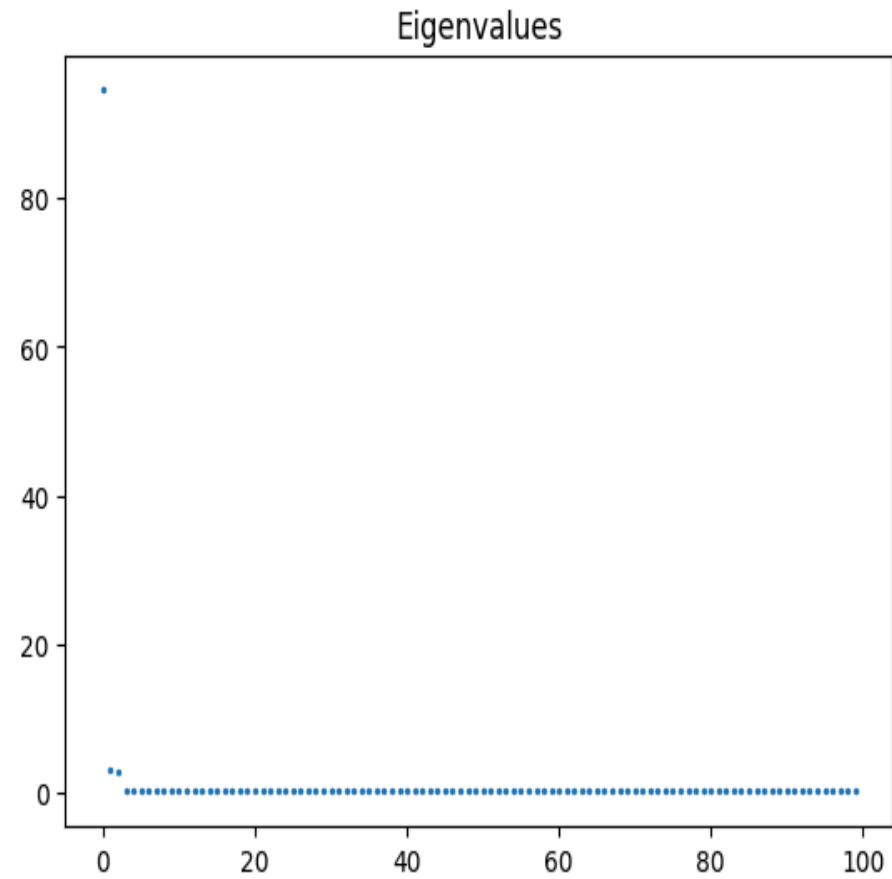
OOPPS! But not all is lost Let's instead consider the Kernel $k' = k + \epsilon I$, with $\epsilon = 0.2$

```
eigvalues,eigenvector=np.linalg.eigh(affinity_matrix)
np.any(eigvalues<0)
```

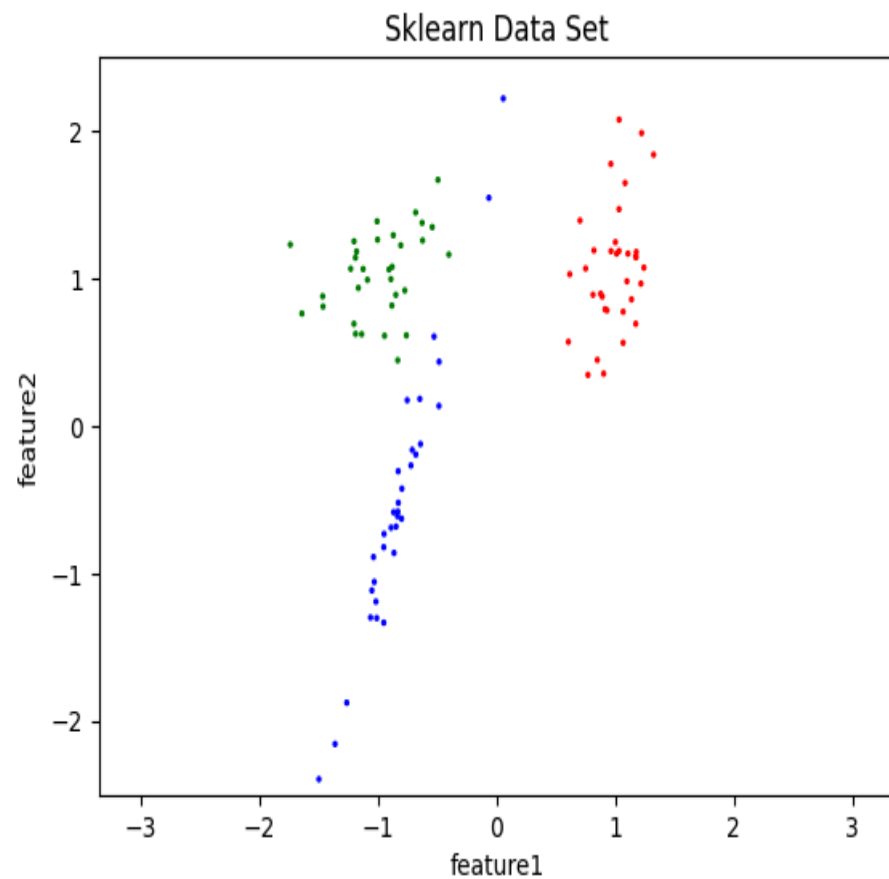
Python

False

It turns out that k' works just as well as k above and the eigenvalues are now non-negative.

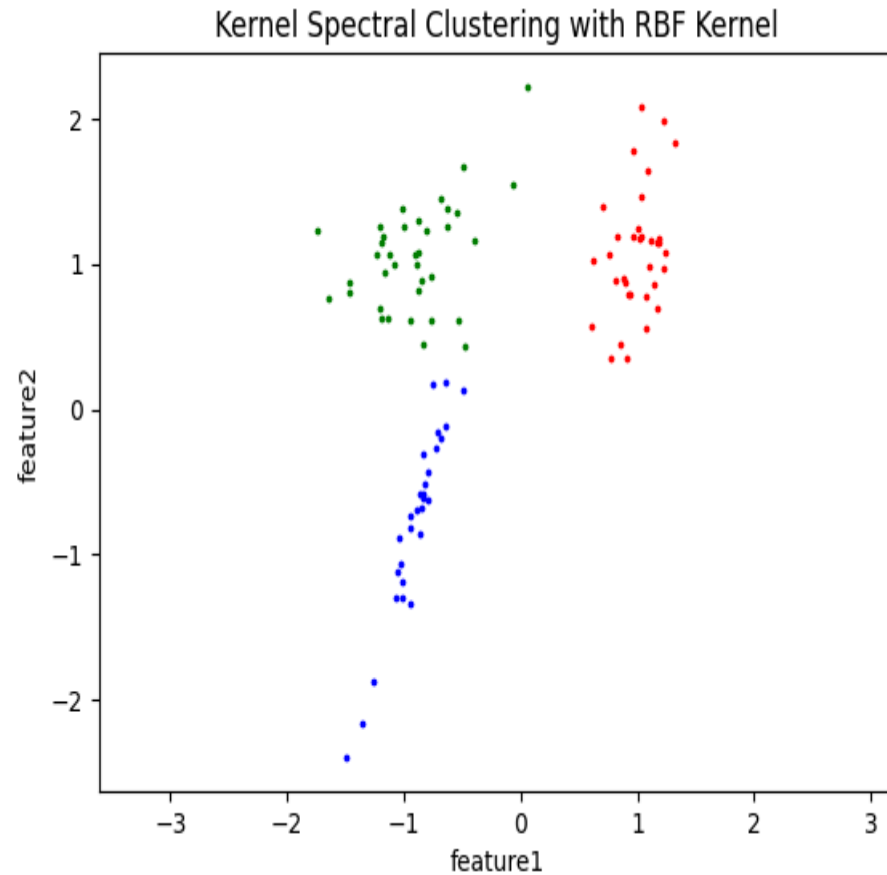


Example3. Consider the following data set from Sklearn



To find a similar grouping based on kernel methods will require two steps.

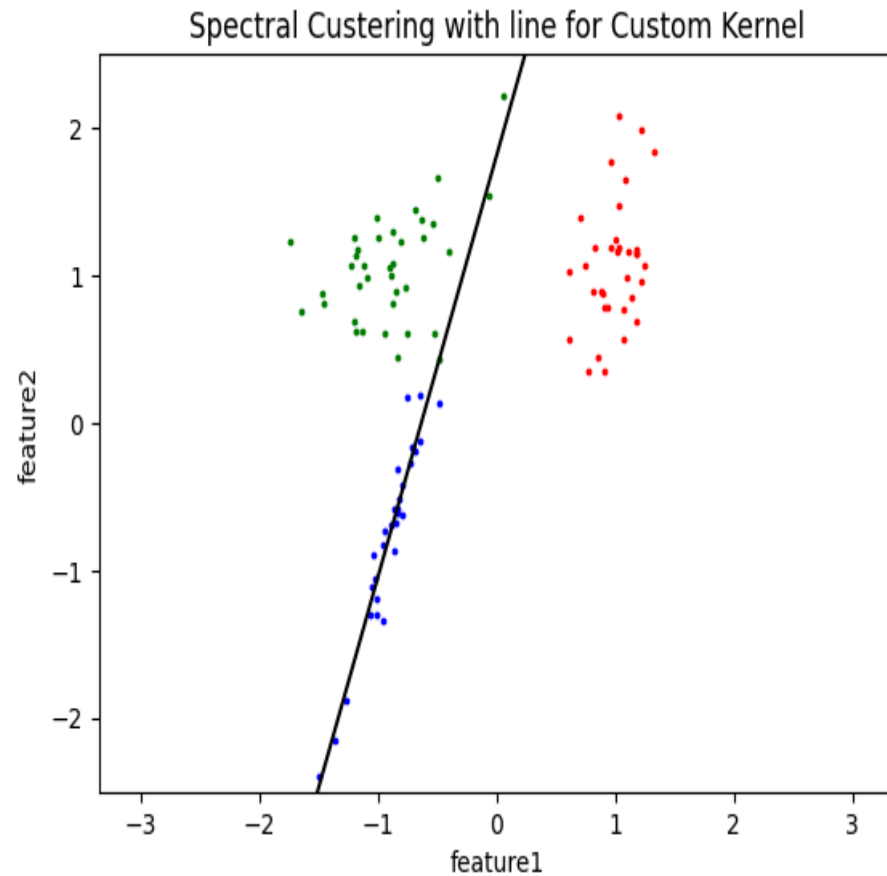
Step1: Initially, group with Spectral Clustering using 'rbf' affinity.



The spectral clustering does a decent job but misses the idea that the middle cluster separates the first and third clusters.

Step2: Follow up with the custom kernel

We can use the results from the spectral clustering to estimate a line that predicts where the middle cluster should extend using linear regression.



Now let's design a kernel which is a function of distance between points, but measured only in the orthogonal direction to this line.

With a little geometry we can see that the following kernel will do just that.

$$k(x, y) = \exp(-\alpha |mx[0] - x[1] - my[0] + y[1]| \sqrt{1 + m^2})$$

where m is the slope of the line and α is an adjustable parameter.

Let's make sure k is p.d.

```
eigvalues,eigenvectors=np.linalg.eigh(affinity_matrix)
np.any(eigvalues<0)
```

Python

False

And here's the finally clustering:

